



F6 快速实现指南 — 按步骤照做

零基础、没做 F3-F5? 没关系。下面每一步都是精确操作，跟着点、跟着设、跟着连，约 60-90 分钟完成。

准备工作

1. 打开 Logisim → File → Save 保存为 `sCPU.circ`
2. `Ctrl+K` 开启自动仿真（或菜单 Simulate → Ticks Enabled）
3. 菜单 Simulate → Tick Frequency → 选 `4 Hz`（慢速便于观察）

记住三个工具：

-  **箭头** = 编辑/连线/移动
-  **手指 (Poke)** = 测试时点击引脚切换 0/1
- 元件库在**左侧面板**，属性在**左下角面板**

第 1 步：搭好全局时钟

#	操作
1.1	左侧 <code>Wiring</code> → 拖放一个 Pin 到画布左上方
1.2	选中 Pin，左下属性： <code>Label = clk</code> , <code>Data Bits = 1</code>
1.3	左侧 <code>Wiring</code> → 拖放一个 Tunnel ，属性 <code>Label = clk</code>
1.4	连线：Pin 的输出 → Tunnel

之后所有需要时钟的地方，都用 Tunnel (`Label= clk`) 连接，避免满屏长线。

第 2 步：搭建 PC（程序计数器）

#	操作
2.1	Memory → 拖放 Register ，属性：Data Bits = 8，Label = PC
2.2	再放一个 Tunnel，Label = clk，连接到 PC 的 clk 端（上方三角）
2.3	再放一个 Tunnel，Label = rst，后面我们接复位（先放这里）

第 3 步：搭建 ROM（指令存储器）

#	操作
3.1	Memory → 拖放 ROM
3.2	属性：Data Bits = 8，Address Bit Width = 3（支持 8 条指令）
3.3	连线：PC 的 Q 输出（低 3 位）→ ROM 的 A（地址）端
3.4	右键 ROM → Edit Contents，先填全 0，后面再改

第 4 步：指令译码

#	操作
4.1	Plexers → 拖放 Decoder ，属性：Select Bits = 2（2→4 译码器）
4.2	从 ROM 的 D 输出是 8 位指令。需要取 opcode（高 2 位）：
4.3	Wiring → 拖放 Splitter ，属性：Fan Out = 4，Bit 0 = bits 0-1，Bit 1 = bits 2-3，Bit 2 = bits 4-5，Bit 3 = bits 6-7
4.4	ROM 的 D → Splitter 的输入

#	操作
4.5	Splitter 的 <code>Bit 3</code> (<code>bits 6-7 = opcode</code>) → Decoder 的 <code>sel</code> 端

Decoder 输出 4 根线，由上到下分别是：

- 第 0 根（最低）= `add` 有效
- 第 1 根 = `io` 有效 ← F6 新增
- 第 2 根 = `li` 有效
- 第 3 根（最高）= `bner0` 有效

第 5 步：搭建 GPR（通用寄存器组）

用 RAM 模拟 4 个 8 位寄存器（2 读口 + 1 写口）。

#	操作
5.1	<code>Memory</code> → 拖放 RAM ，属性： <code>Data Bits = 8</code> ， <code>Address Bit Width = 2</code>
5.2	但这只有 1 读口，我们需要 2 读口。所以拖放两个 RAM ，地址位宽都是 2：
5.3	第一个 RAM：Label = <code>GPR</code> （当作主存储，负责写入 + 读口1）
5.4	第二个 RAM：Label = <code>GPR_r2</code> （只负责读口2，两个 RAM 内容需保持同步）

⚠ **双 RAM 同步是关键**：两个 RAM 的 `D`（数据输入）、`A`（写地址）、`WE`、`clk` 必须接相同信号。读地址可以不同（分别为 `raddr1` 和 `raddr2`）。

GPR 接线：

信号	连接目标
GPR.clk	clk Tunnel
GPR_r2.clk	clk Tunnel
GPR.D（数据输入）	→ 来自 GPR wdata Mux 的输出（第 8 步）

信号	连接目标
GPR_r2.D	同上
GPR.A (写地址)	→ 来自 rd 字段 (Splitter 的 bits 4-5, 即 rd[1:0])
GPR_r2.A (写地址)	同上
GPR.WE	→ 来自 wen 逻辑 (第 7 步)
GPR_r2.WE	同上
GPR.A (读地址)	→ raddr1 (第 11 步)
GPR_r2.A (读地址)	→ raddr2 = rs2 (Splitter 的 bits 1-0 那两根? 不对...)

🔧 **纠正**: 标准 RAM 只有一个地址端口 (A), 读写共用。我们需要的是: 写地址 = rd, 读地址分别 = raddr1 和 raddr2。但 Logisim RAM 只有一个 A 端口.....

解决方法: 使用 **Register** 而非 RAM 来搭建 GPR, 用 4 个 8 位 Register + Mux 实现。这是 F5 原始指南推荐的做法。

重新设计 GPR (用 Register + Mux, 更简单)

#	操作
5a.1	Memory → 拖放 4 个 Register , 各自属性 Data Bits = 8, Label 分别 = R0, R1, R2, R3
5a.2	全部 clk 端接 clk Tunnel
5a.3	Plexers → 拖放 2 个 Multiplexer (4 选 1), 属性: Data Bits = 8, Select Bits = 2
5a.4	第一个 Mux (读出 raddr1): 4 个数据端分别接 R0.Q, R1.Q, R2.Q, R3.Q
5a.5	第二个 Mux (读出 raddr2): 4 个数据端也分别接 R0.Q, R1.Q, R2.Q, R3.Q
5a.6	Plexers → 拖放 1 个 Decoder (2→4), 作为「写地址译码器」
5a.7	写地址译码器输入 = rd 字段 (Splitter 的 bits 4-5), 输出 4 根线

#	操作
5a.8	每个 R[x] 的 WE 端 = (wen) AND (写地址译码器第 x 根输出)
5a.9	4 个 Register 的 D 端都接到 GPR wdata Mux 的输出

这样：读端口 = 2 个独立的 Mux (raddr1 选通、raddr2 选通)，写端口通过 Decoder 选择写入哪个寄存器。

第 6 步：操作数字段抽取

之前第 4 步已将指令用 Splitter 分成 4 段。现在细化：

#	操作
6.1	确认 Splitter 的 Fan Out = 4，排列为：[1:0] [3:2] [5:4] [7:6]
6.2	Bit 0 (bits 1-0) = imm 的低 2 位 (也是 rs2/idx)
6.3	Bit 1 (bits 3-2) = imm 高 2 位 (也是 rs1/s 字段)
6.4	Bit 2 (bits 5-4) = rd 字段
6.5	Bit 3 (bits 7-6) = opcode (已接 Decoder)

不同指令复用相同位域。例如 add 的 rs2 在 bits 1-0，io 的 idx 也在 bits 1-0，共用 Splitter 的 Bit 0。

第 7 步：控制逻辑 — wen (GPR 写使能)

指令	wen
add	1
li	1

指令	wen
io 输入 (i/o=0)	1
io 输出 (i/o=1)	0
bner0	0

#	操作
7.1	Wiring → 拖放 1 个 NOT 门 (用于 i/o 取反得到 i/o==0)
7.2	Splitter 的 Bit 1 是 bits 3-2, 其中 bit 2 = i/o (对 io 指令而言)。需要单独再拆: 再放一个 Splitter: Fan Out=2, Bit 0 = 1bit(b3), Bit 1 = 1bit(b2=i/o)
7.3	NOT 门输入接 i/o 位, 输出 = i/o==0
7.4	Gates → 拖放 2 个 AND 门, 1 个 OR 门 (3 输入不够用 2 级联)
7.5	AND1: io_v (Decoder 第 1 线) AND NOT(i/o) → io_in_valid
7.6	OR1: add_v (Decoder 第 0 线) OR li_v (Decoder 第 2 线)
7.7	OR2: OR1 的输出 OR io_in_valid → 最终 wen
7.8	最终 wen → 接 4 个 AND 门 (与写地址译码器输出配合后接各 R[x].WE)

第 8 步：控制逻辑 — GPR wdata (写入数据来源)

指令	wdata 来源
add	加法器结果
li	s 移位后的立即数
io 输入	设备输入数据

#	操作
8.1	Plexers → 拖放 2 个 Mux (2选1, 8位) 做级联
8.2	Mux_A: sel = li_v, 数据0 接 Mux_B 输出, 数据1 接 li 移位结果 (第 9 步)
8.3	Mux_B: sel = io_in_valid (io_v AND NOT(i/o)) , 数据0 接 加法器输出, 数据1 接 设备输入
8.4	Mux_A 输出 → 4 个 R[x].D

第 9 步: li 立即数生成器

#	操作
9.1	Plexers → 拖放一个 Mux (4选1, 8位, Select Bits=2)
9.2	Mux 的 sel 端 = Splitter 的 bits 3,2 中的 s 字段 (注意: s 在 bits 3-2, 对 li 而言)
	si 字段 = Splitter bit 1 (bits 3-2) , 其中高位是 s[1], 低位是 s[0]
9.3	4 个数据端分别接 4 个 Constant :

sel(s)	Constant 值 (8位)	含义
00	0x03	00000011
01	0x0C	00001100
10	0x30	00110000
11	0xC0	11000000

⚠ 这是硬编码 imm=11 的版本! 灵活版本见下方。

灵活版（推荐，支持任意 imm）

#	操作
9a.1	Arithmetic → 拖放 Shifter （不是 Barrel Shifter），Data Bits = 8
9a.2	Wiring → 拖放 Bit Extender ，Input=2, Output=8, Type=Zero
9a.3	Bit Extender 输入 = Splitter Bit 0（imm, bits 1-0），输出 → Shifter.Data
9a.4	Shift 量 = $s \times 2$ 。用拼接方式：放一个 Splitter，将 s[1:0] 后面拼一个 0 = 3 位
	Splitter3: Input 是 3 根线（s[1], s[0], 固定0），Fan Out=1，输出 3 位
9a.5	这个 3 位输出接 Shifter 的 Shift 端
9a.6	Shifter 输出 = li 移位结果 → 接第 8 步的 Mux_A 数据 1 端

第 10 步：加法器 + 比较器

#	操作
10.1	Arithmetic → 拖放 Adder ，属性 Data Bits = 8
10.2	加法器 A 端 ← rdata1 Mux 的输出（GPR 读口 1）
10.3	加法器 B 端 ← rdata2 Mux 的输出（GPR 读口 2）
10.4	加法器输出 → 第 8 步 Mux_B 的数据 0 端
10.5	Arithmetic → 拖放 Comparator ，属性 Data Bits = 8
10.6	A 端 = GPR 读口 1（设为读 R[0] 即 raddr1=0），B 端 = GPR 读口 2（R[rs2]）
10.7	比较器 $\neq$ 输出 → 用于 bner0 判断

第 11 步：控制逻辑 — GPR raddr1

#	操作
11.1	<code>Plexers</code> → 拖放一个 Mux (4选1, 2位, Select Bits=2)
11.2	数据 0 = Splitter Bit 1 (bits 3-2 = rs1)
11.3	数据 1 = Constant 0 (2位, R[0])
11.4	数据 2 = Splitter Bit 2 (bits 5-4 = rd)
11.5	数据 3 = 无关 (也接 0)
11.6	raddr1 Mux 的输出 → GPR 读口 1 Mux 的 sel (即 4 选 1 Mux 去读 R[x])

sel 逻辑 (用两个控制线) :

- 选 rs1: add_v 有效时
- 选 0: bner0_v 有效时
- 选 rd: io_v 有效时
- 只需要 2 位选择信号, 此处用组合逻辑实现或用 4 选 1 Mux 的 sel 直接接

简化做法: raddr1 Mux 用 2 位 sel, sel[0] = bner0_v OR io_v, sel[1] = io_v:

sel[1:0]	选通	适用指令
00	rs1	add, li
01	0 (R[0])	bner0
10	rd	io

用一个小的组合逻辑生成 sel:

- sel[0] = bner0_v
 - sel[1] = io_v
-

第 12 步：控制逻辑 — PC 更新

#	操作
12.1	Arithmetic → 拖放 Adder , 属性 Data Bits = 8
12.2	加法器 A 端 = PC.Q (当前 PC 值)
12.3	Plexers → 拖放 Mux (2选1, 8位)
12.4	Mux 数据 0 = Constant 1 (8 位) , Mux 数据 1 = sign_ext(offset)
12.5	Mux 输出 → 加法器 B 端
12.6	Mux sel = bner0_v AND (比较器#输出)
12.7	加法器输出 → PC.D
12.8	bit Extender (4→8, Sign Extend) , 输入 = offset (Splitter 的 bits 5-2) , 输出接 Mux 数据1

offset 是 bits 5-2 (高 4 位中的低 4 位) 。需要用 Splitter 重新分组: opcode[7:6] 已经用了, 从剩余 6 位中取 offset[5:2]。用另一个 Splitter 从指令中分离 offset。

指令位域重新整理 (建议新建一个 Splitter) :

字段	位
opcode	7-6
rd	5-4
杂项	3-2 (s / offset高2位 / rs1)
杂项	1-0 (imm / offset低2位 / rs2 / idx)

对于 bner0: offset = bits 5-2。

用 4 根线单独引出: bits 5,4,3,2 → Bit Extender (4→8, Sign Extend) 。

第 13 步：输出通路 — LED Bar

#	操作
13.1	Input/Output → 拖放 LED Bar
13.2	属性：Input Format = One Wire , Segments = 8
13.3	Memory → 拖放 Register （输出锁存器），属性 Data Bits = 8
13.4	锁存器的 D = rdata1 Mux 输出（见下方注意）
13.5	锁存器的 WE = io_v AND i/o（i/o=1时使能）
13.6	锁存器的 clk = clk Tunnel
13.7	锁存器的 q → LED Bar 输入

注意：io 输出时需要读出 R[rd]。在第 11 步中，raddr1 已支持 io_v 时选 rd，所以 rdata1 Mux 输出就是 R[rd] 的值，直接接到输出锁存器的 D。

第 14 步：输入通路

#	操作
14.1	Wiring → 拖放 Constant ，属性 Data Bits = 8, Value = 0x00
14.2	这个 Constant 模拟输入设备 0
14.3	Constant 输出 → 第 8 步 Mux_B 的数据 1 端（设备输入数据）

测试时修改此 Constant 的值即可模拟不同输入。

第 15 步：编写测试程序

右键 ROM → **Edit Contents**，填入以下十六进制值：

地址	值	指令说明
00	90	li r1, s=00, imm=00 → r1=0
01	A1	li r2, s=00, imm=01 → r2=1
02	BE	li r3, s=11, imm=10 → r3=10
03	16	add r1, r1, r2 → r1+=r2
04	29	add r2, r2, 01 → r2+=1 (01=imm=1,s=00)
05	F7	bner0 offset=-3, r3 → if r1≠r3, PC跳回
06	54	io r1, i/o=1, dev=0 → 输出r1到LED
07	00	(填充, halt)

机器码推算参考

指令	二进制
li r1, s=00, imm=00	10_01_00_00 = 0x90
li r2, s=00, imm=01	10_10_00_01 = 0xA1
li r3, s=11, imm=10	10_11_11_10 = 0xBE
add r1, r1, r2	00_01_01_10 = 0x16
add r2, r2, 01	00_10_10_01 = 0x29
bner0 offset=-3, r3	11_1101_11 = 0xF7
io r1, OUT, dev0	01_01_1_00 = 0x54

第 16 步：运行 + 调试

1. `Ctrl+K` 启动仿真
2. `Ctrl+R` 复位

3. 在关键节点放 **Probe** (Wiring 分类) 观察值:
- PC.Q — 看当前执行到哪条指令

• ROM.D — 看当前取出的指令

• R1.Q, R2.Q, R3.Q — 看寄存器值

• LED Bar — 看最终输出
4. 逐步检查:
- 前 3 个 cycle: li 指令写入 R1/R2/R3

• 之后循环: R1 每次增加 (1, 3, 6, 10, 15, ... 直到 55)

• 最后 io 指令: LED Bar 亮起 00110111

快速排查

现象	可能原因
什么都不工作	<code>Ctrl+K</code> 开了吗? 时钟连了吗?
红色线	两处输出驱动同一根线
蓝色线	信号没有驱动源 (忘记接线)
橙色线	位宽不一致 (如 4 位接了 8 位口, 用 Bit Extender)
PC 卡住不动	PC 更新 Mux 选择有问题, 或时钟没接
LED 全不亮	输出锁存器 WE 不对, 检查 <code>io_v AND i/o</code>
bner0 跳错	offset 符号扩展方向检查, offset 字段位域确认

版本选择

上面描述的是「Register 版 GPR」的做法。如果你想用 RAM 做 GPR (更接近真实 CPU) , 需要处理单端口 RAM 的问题, 方法是在同一个 cycle 内分时复用, 但 Logisim 中较复杂。Register 方案最简单, 推荐首次搭建使用。